

90 Days SDET Interview Prep

Day 6 (Reflection)

Contents

What is Reflection in Java and why do we need it?	2
What are the 3 ways to get a Class object Reference?	4
How to access private fields using Reflection?	6
How to create objects without using new keyword?	7
What are the disadvantages of Reflection?	8

What is Reflection in Java and why do we need it?

Reflection is a Java API that allows you to **inspect and manipulate classes, methods, fields, and constructors at runtime** even if you don't know their names at compile time.

Reason	Problem	Solution
Access Private Members	Can't access private variables/methods normally	You can use Reflection to bypass it and access private variables and methods. Used for debugging
Dynamic Method Calls	You cant call method name stored in a variable; <code>String method= "doLogin"</code>	You can use Reflection to invoke it by string name.
Dynamic Object Creation	Can't create object when class name is String <code>String className="LoginPage"</code>	Reflection can instantiate.

Key Points to Remember:

1. **Reflection** = Runtime manipulation
2. Used heavily by frameworks (**TestNG, Selenium Webdriver, Spring, Hibernate**)
3. **Package:** java.lang.reflect
4. **Main Classes:** Class, Method, Field, Constructor

Code Example:

```
public class Runner {
    public static void main(String[] args) {
        Cal c = new Cal();
        System.out.println(c.add(10, 20));
        Class<?> clazz = Cal.class; //Gets the Class object representing
        Cal class
        //Class<?> - Generic type (? means any class)

        /*
            clazz contains metadata about Cal class:
            - Class name: "Cal"
            - Package: "com.day7"
            - All methods: add(), demo()
            - All fields: (if any)
            - Constructors
        */

        Method method1 = clazz.getMethod("demo");
        method1.invoke(c);

        Method method2 = clazz.getMethod(c,"add", int.class, int.class);
        //Search for a public method in Cal class
        //Method name to find
        //Parameter types (two int parameters)
        int result = (int) method.invoke(c, 10, 20);
        System.out.println(result);

        //for static method you dont need to pass any object reference
        Method method3 = clazz.getMethod("hello");
        method3.invoke(null);
    }
}
```

What are the 3 ways to get a Class object Reference?

Ans:

```
// 1. REGULAR OBJECT
Cal c = new Cal();
// 'c' is a regular object
// Contains: actual data and methods you can use

// 2. CLASS OBJECT
Class<?> clazz = Cal.class;
// 'clazz' is a Class object (special object)
// Contains: metadata/information ABOUT the Cal class
```

- What's the Purpose of Class Object:
 - To **INSPECT** the class [*constructor, private/public/static variables and methods*]
 - Contains: Metadata (information about the class)

There are 3 ways to obtain a Class object in Java

1. .class literal (Compile time)

```
Class<?> clazz = Cal.class;
```

When to use:

- ✓ You know the class name at **compile time**
- ✓ Most common and recommended
- ✓ No object needed
- ✓ No exceptions

2. getClass() (RunTime)

```
Cal c = new Cal();  
Class<?> clazz = c.getClass();
```

When to use:

- ✓ You already have an object instance
- ✓ Want to get class info from existing object
- ✓ No exceptions

3. Class.forName() (dynamic):

```
Class<?> clazz = Class.forName("com.day7.Cal");
```

When to use:

- ✓ Class name is a String (from config/Excel/database)
- ✓ Loading classes dynamically at runtime
- ✓ Throws ClassNotFoundException
- ✓ Must use fully qualified name

How to access private fields using Reflection?

Steps:

- ✓ `getDeclaredField()` for private fields
- ✓ `setAccessible(true)` to bypass private access
- ✓ `field.get(object)` to get value
- ✓ `field.set(object, value)` to set value.

```
public class Cal {  
  
    // Private field - cannot be accessed directly  
    private String calculatorName = "Scientific Calculator";  
  
}
```

```
Class<?> clazz = Cal.class;  
  
Field field = clazz.getDeclaredField("calculatorName");  
  
//Bypass private access control  
field.setAccessible(true); // ← IMPORTANT  
  
// Get the value  
String calculatorName = (String) field.get(c);  
  
System.out.println("Private field value: " + calculatorName);
```

For Methods:

```
Method multiplyMethod = clazz.getDeclaredMethod("multiply", int.class, int.class);  
multiplyMethod.setAccessible(true);  
  
int result1 = (int) multiplyMethod.invoke(c, 5, 3);  
  
System.out.println("multiply(5, 3) = " + result1);
```

How to create objects without using new keyword?

```
Class<?> clazz1 = Cal.class;
Cal c1 = (Cal) clazz1.newInstance(); // Deprecated in Java 9+

Class<?> clazz2 = Cal.class;
Constructor<?> constructor = clazz2.getDeclaredConstructor();
Cal c2 = (Cal) constructor.newInstance(); // Recommended way

String className = "com.day7.Cal"; // From config/Excel
Class<?> clazz3 = Class.forName(className);
Constructor<?> constructor3 = clazz3.getDeclaredConstructor();
Cal c3 = (Cal) constructor3.newInstance();
```

What are the disadvantages of Reflection?

- Reflection is 2-3x slower than direct method calls.

Why slower?

- Extra steps: Find method → Check access → Invoke
- JVM can't optimize reflection calls like direct calls
- Type checking happens at runtime, not compile-time

- Breaks Encapsulation

Problem: Can access and modify private members, violating OOP principles.

Why this is bad:

- Bypasses validation logic
- Violates "private" contract
- Makes code unpredictable and unsafe

- No Compile-Time Type Safety

Problem: Errors appear at runtime, not compile time. Harder to catch bugs.

Impact:

- Bugs discovered late (during testing or production)
- More difficult debugging
- No IDE auto-complete help
- Typos not caught until runtime